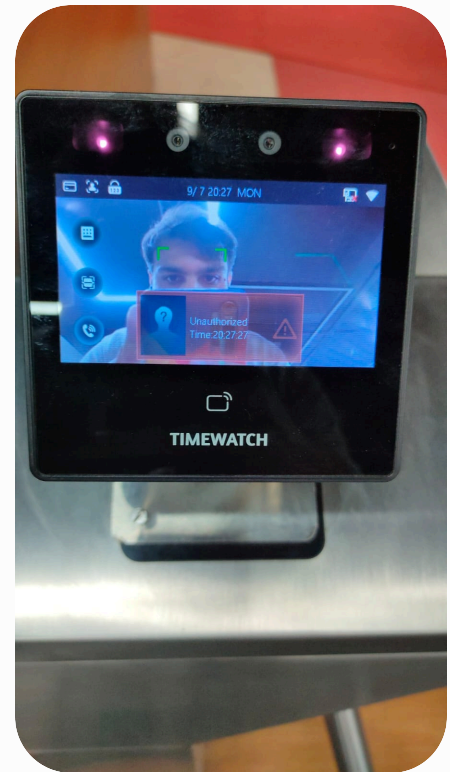


Today 9:17 PM



problem: this hardware sends its data to iAS (integrated attendance system) software. The software is insufficient to cater to modern requirements such as sending notifications, access through mobile device. Role based access to view/modify data. modifying the data incorrectly can also cause issues which require major efforts to mitigate the database issues and bring it back to function properly.

ask: If we can capture the data this hardware sends via an intermediate application and create our own application that supports above + more features.

what I want to know: is it possible to access the data that this hardware is sending? Is it allowed? is it legal? even if it is possible, can it void the warranty? any other challenges?

Show less ^



TrueFace3000-Datasheet...
PDF

The issue is that if I maintain a different database, I won't be able to provide certain features like membership extension. If I extend the membership or cancel it, hardware should function as per this data. If hardware is only connected with iAs and iAs database is not up-to-date, then it's useful to provide these features.

Yes — **this changes the architecture significantly**, and your concern is correct.

If your application has a completely separate database that is only a **copy of iAS attendance data**, then you cannot reliably implement features such as:

"Member's subscription expired → immediately prevent that person from entering."

because the **device ultimately needs to know whether that person is authorized**.

The good news is that the **TrueFace3000 datasheet you provided makes this approach considerably more promising than I initially thought**.

The key discovery in your datasheet

The TrueFace3000 is not merely an attendance reader. It is a **standalone access-control device**, and the datasheet explicitly says it supports:

- TCP/IP and Wi-Fi
- Network Update

- Remote Verification
- Restricted List / Trusted List
- Web Configuration
- Real-time monitoring
- Lock control
- **SDK and API**
- 5,000 users/faces/cards/passwords
- 300,000 logs  TrueFace3000-Datasheet-Ver.2.0 +1

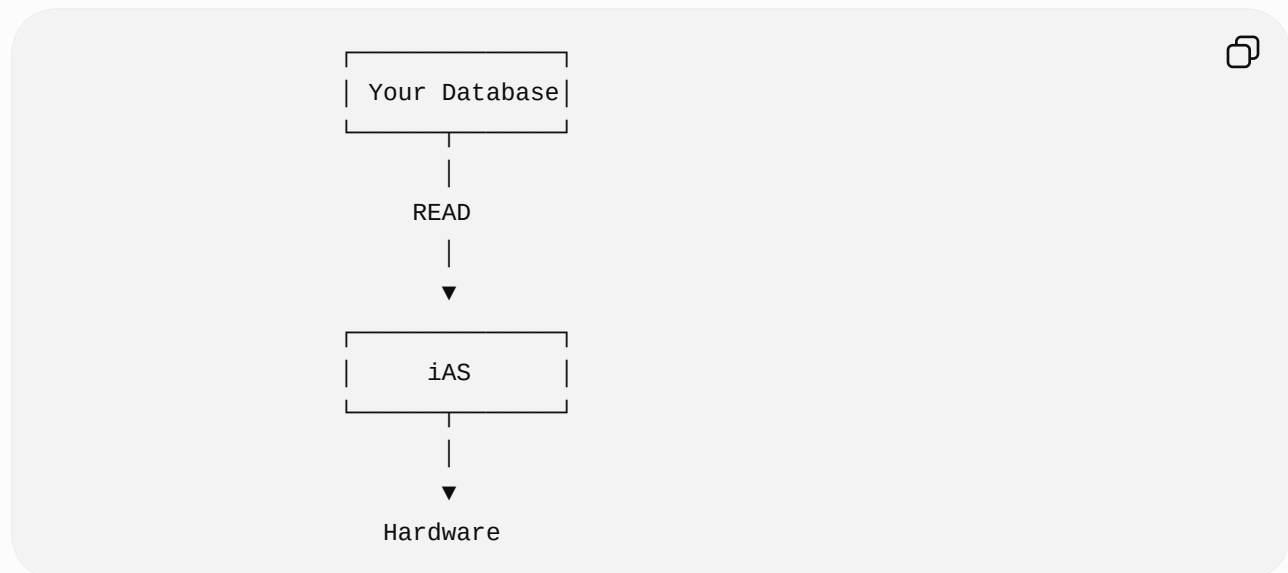
TimeWatch's current product page also describes the TrueFace3000 as supporting access control and network/software integration.  TimeWatch

So I would **change my recommendation** from "build a read-only layer over iAS" to:

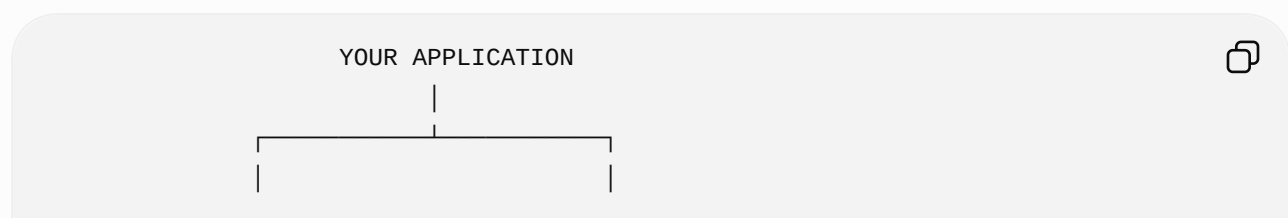
Build your own system as the authoritative membership/access-control system, and use iAS only as an optional compatibility/integration layer.

The architecture I would pursue

Instead of:

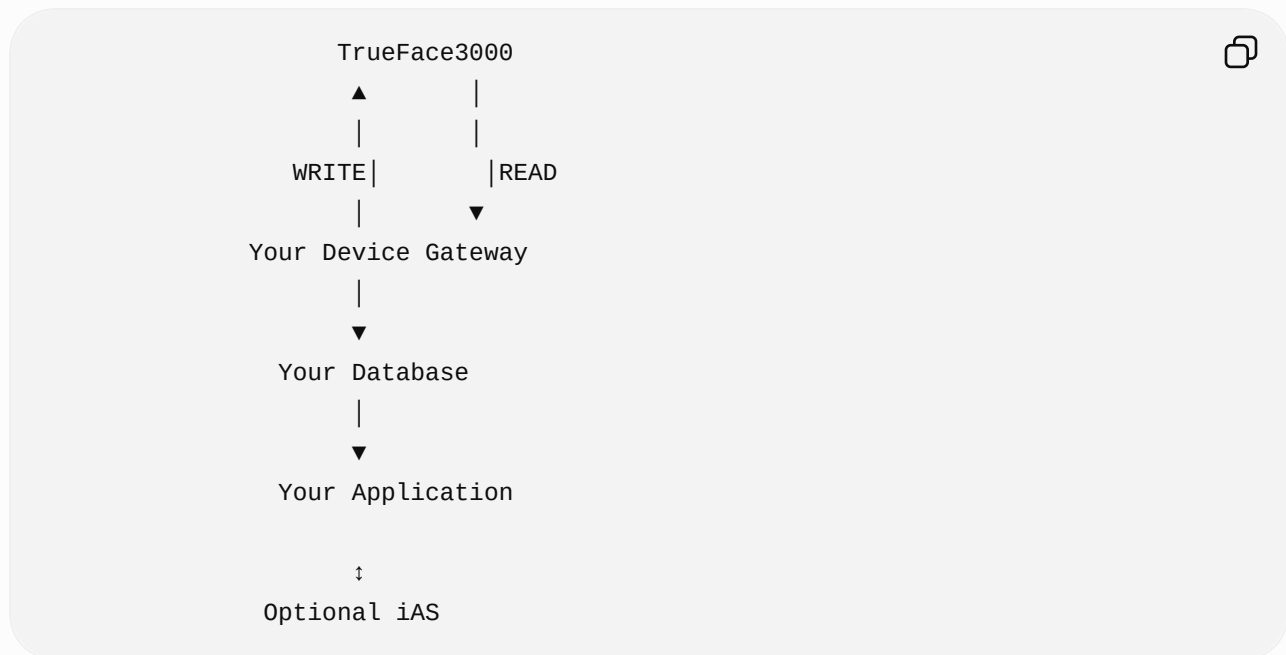


I would aim for:





And **iAS** becomes secondary:



Your membership-extension example

Suppose a gym member has:

Rahul
Membership:
01 Jan → 30 Sep

Your application stores:

```
member_id = 1234  
valid_from = 2026-01-01  
valid_until = 2026-09-30  
status = ACTIVE
```



On September 30:

Your application



```
graph TD  
A[Your application] --> B[Membership expires]  
B --> C[Device Gateway]  
C --> D[TrueFace3000]  
D --> E[Disable / restrict member]
```

Now Rahul approaches the device on October 1:

Face detected



```
graph TD  
A[Face detected] --> B[Device identifies Rahul]  
B --> C[Authorization]  
C --> D[❌ Access denied]
```

That's fundamentally different from merely copying attendance records.

And when Rahul renews?

Suppose he pays ₹10,000 and renews until December 31.

Your system does:

Payment successful



↓

```
Membership updated
  ↓
Membership ACTIVE
  ↓
Device authorization updated
  ↓
TrueFace3000
  ↓
Rahul can enter
```

No iAS database modification is required.

This is the architecture I'd prefer.

But there's an important technical question

We still need to determine **exactly what the TrueFace3000 API/SDK allows us to control.**

The datasheet confirms:

SDK and API — Yes

and also:

Network Update — Yes

Remote Verification — Yes

Restricted List/Trusted List — Yes

Web Configuration — Yes

 TrueFace3000-Datasheet-Ver.2.0

But the datasheet **does not tell us the actual API endpoints/functions.**

For example, we need to know whether the SDK/API allows:

```
Create user
Delete user
Update user
Enable user
Disable user
Set validity start
Set validity end
Set access schedule
Set restricted list
```



```
Read attendance logs
Clear attendance logs
Push face template
Delete face template
Get device status
```

The datasheet alone doesn't establish all of these.

So **the next critical artifact is the TrueFace SDK/API documentation**, not the iAS database.

TimeWatch's current downloads page actually lists both a **TrueFace SDK** and **Bio Series SDK**, which is encouraging.  TimeWatch

There are actually two separate data flows

This is something I'd strongly recommend keeping conceptually separate.

1. Attendance

```
Person
↓
TrueFace3000
↓
"Shivam entered at 09:02"
↓
Your system
```



Your system records the event.

2. Authorization

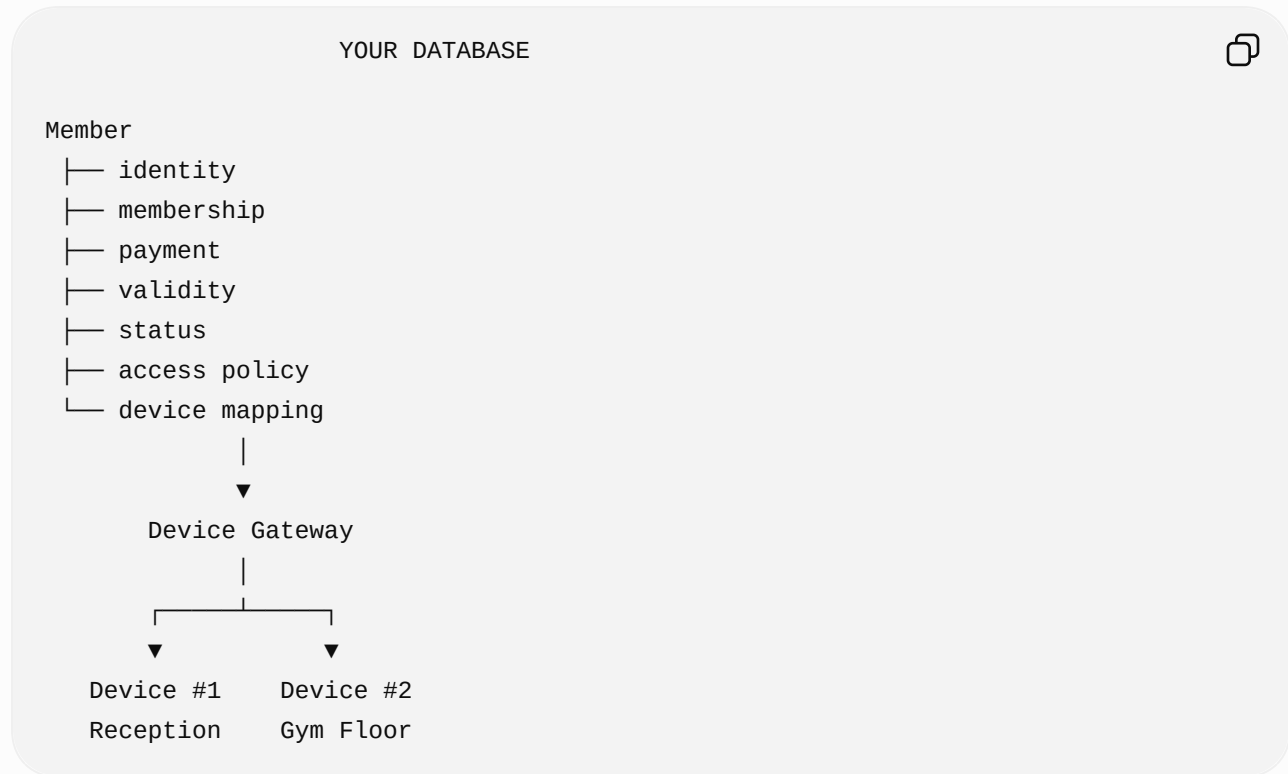
```
Your membership system
↓
"Shivam is active until 31 Dec"
↓
Device Gateway
↓
TrueFace3000
↓
Device knows Shivam is authorized
```



This second flow is what solves your problem.

This also means your database can safely be the "source of truth"

For example:

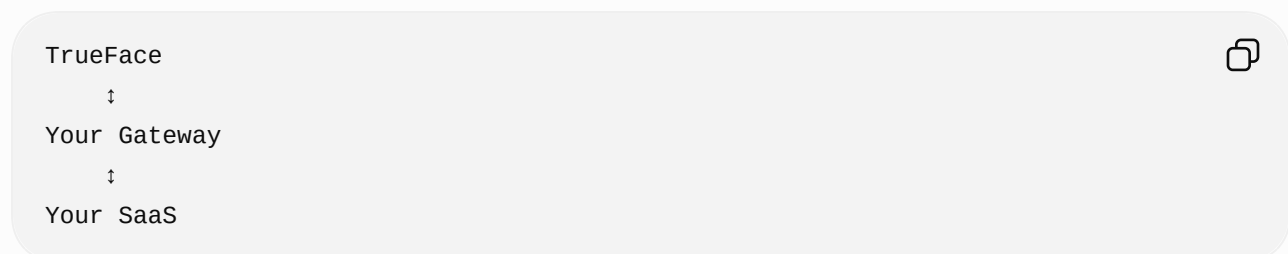


You don't need to manipulate iAS at all.

What about iAS?

You have two choices.

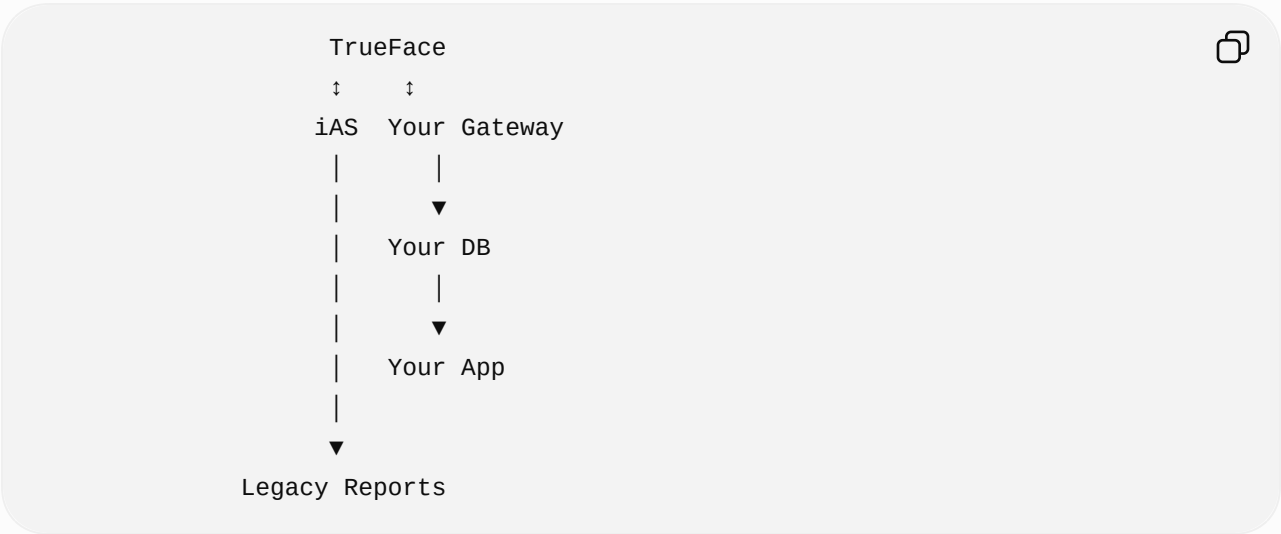
Option 1 — Eventually eliminate iAS



This is the cleanest architecture.

Option 2 — Keep iAS running

This might be preferable during migration:

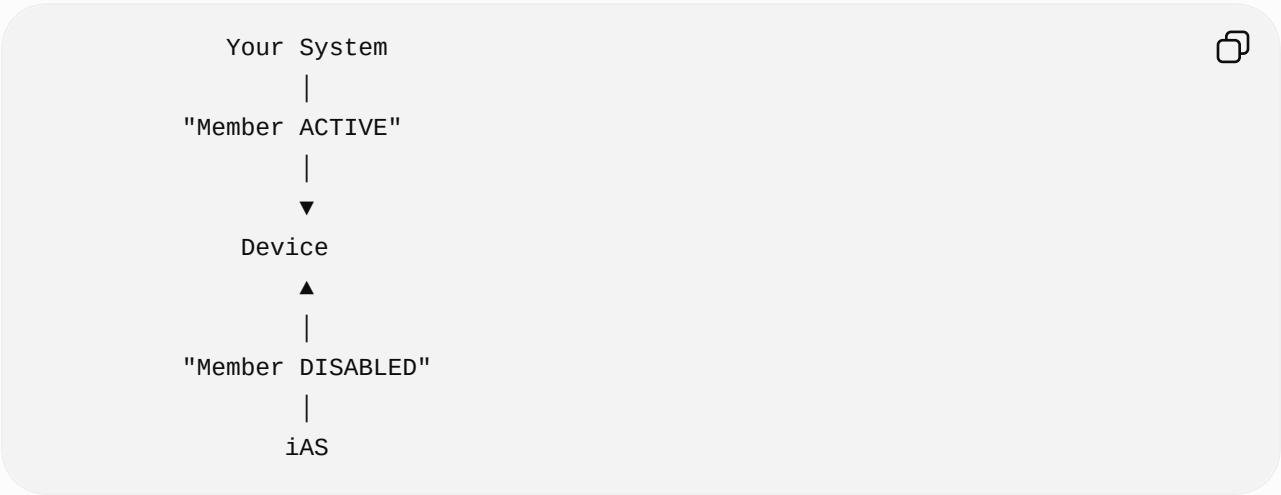


But I would avoid having both iAS and your system independently changing device authorization.

That's where trouble starts.

The biggest danger: two masters

Imagine:



Now what happens?

You have a synchronization conflict.

Therefore, we need to establish:

Who is the authoritative controller of the device?

If you're building the new system, I'd make:

Your system = Master

```
graph TD
    A[Your Database] --> B[Device Gateway]
    B --> C[TrueFace]
```



and:

iAS = legacy/read/reporting system

not another authority.

There's another very important possibility

Because the device itself supports **300,000 records**, we don't necessarily need to constantly communicate with it for attendance.  TrueFace3000-Datasheet-Ver.2.0

We can design for temporary network failures.

For example:

```
graph TD
    A[INTERNET DOWN] --> B[Device still works]
    B --> C[Local authentication]
    C --> D[Attendance stored]
    D --> E[Network returns]
    E --> F[Sync Gateway]
    F --> G[Your Database]
```



That is extremely important for a gym/business.

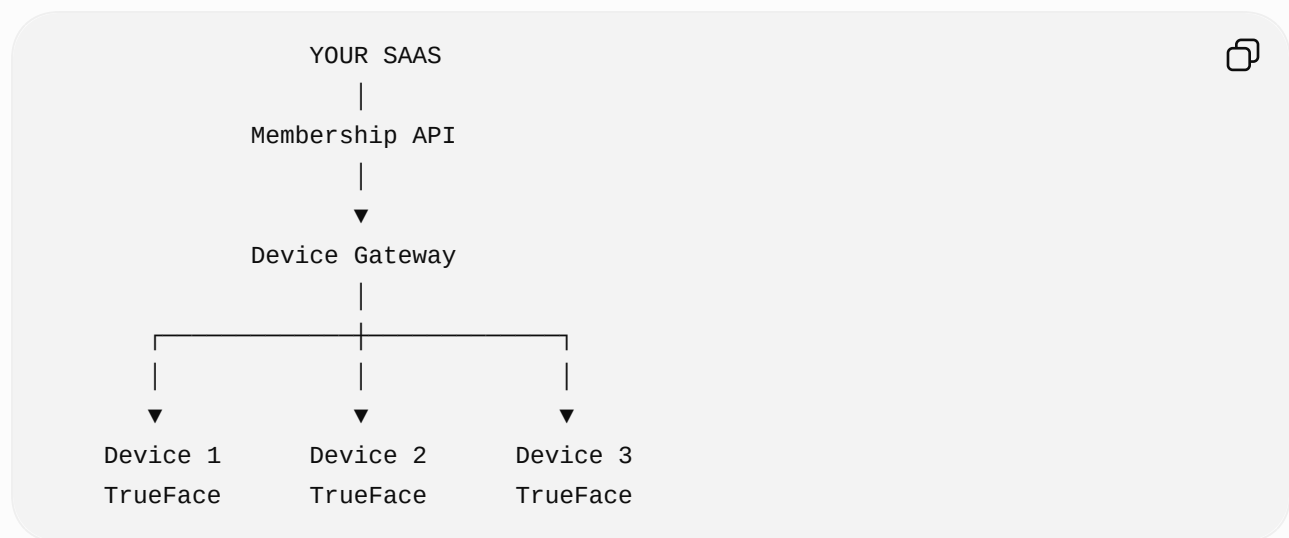
You don't want:

"Internet is down, therefore nobody can enter."

The exact offline authorization behavior needs to be verified against the SDK/device documentation, but the device's standalone architecture and local capacity make this an important design consideration. [TrueFace3000-Datasheet-Ver.2.0](#)

I would therefore design a "Device Gateway"

This is probably the most important component of the entire project.



The gateway handles:

Commands

CREATE_MEMBER
UPDATE_MEMBER
DISABLE_MEMBER
ENABLE_MEMBER
EXTEND_MEMBERSHIP
REMOVE_MEMBER
UPDATE_ACCESS_POLICY

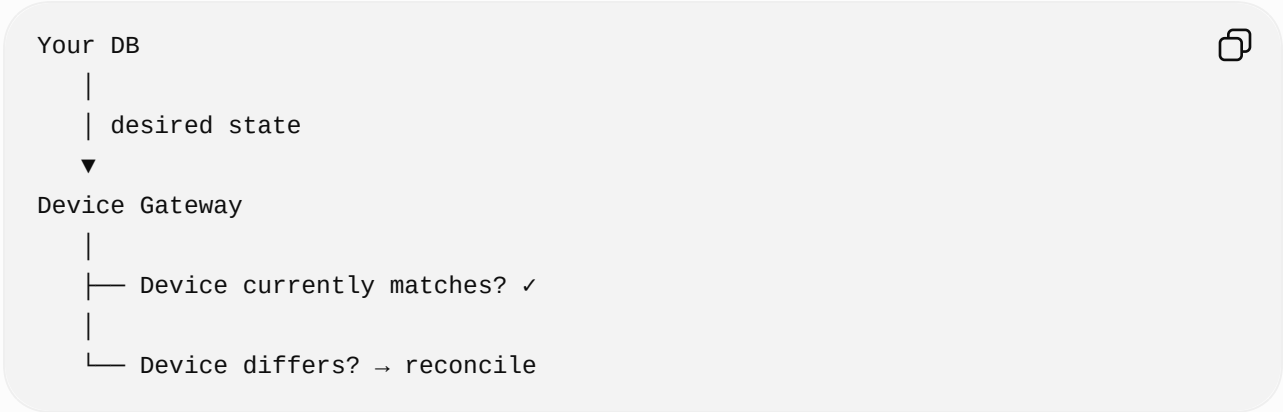
Events

ACCESS_GRANTED
ACCESS_DENIED
ATTENDANCE_IN

ATTENDANCE_OUT
DEVICE_OFFLINE
DEVICE_ONLINE
SYNC_FAILED

Synchronization

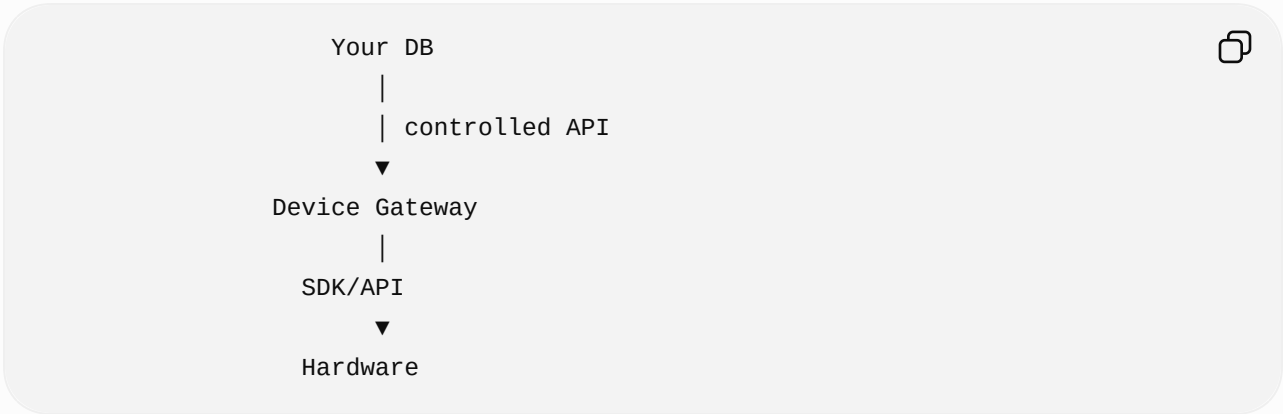
And critically:



This also solves your database-corruption concern

You mentioned that changing iAS data incorrectly can cause serious database issues.

With this architecture:



Nobody needs to directly edit:

iAS database ❌

Your application can enforce:

RBAC
+

```
validation
+
transactions
+
audit trail
+
approval workflow
+
idempotency
+
rollback/reconciliation
```

For example:

"Extend membership"

isn't:

</> SQL



It's:

```
Membership Service
↓
Validate payment
↓
Update membership
↓
Generate DeviceAuthorizationCommand
↓
Gateway
↓
Device
↓
ACK
↓
Mark synchronization successful
```



Much safer.

One thing I would investigate immediately

Can the TrueFace3000 itself enforce a validity period for a user?

The datasheet says it supports **128 "Period eList" periods, 128 holiday periods**, network update, restricted/trusted lists, and remote verification.  TrueFace3000-Datasheet-Ver.2.0

That is very promising.

If the SDK allows something like:

```
User 1234
Valid From: 01-09-2026
Valid To:   30-09-2026
Period:     Gym Members
```



then we have an excellent solution.

Your membership system becomes:

```
Membership expires
↓
Change device authorization
↓
Device itself rejects access
```



rather than:

```
Member recognized
↓
Your server must respond
↓
Allow / deny
```



The former is usually **much more robust** for a physical-access system.

One more thing: your product idea becomes much stronger

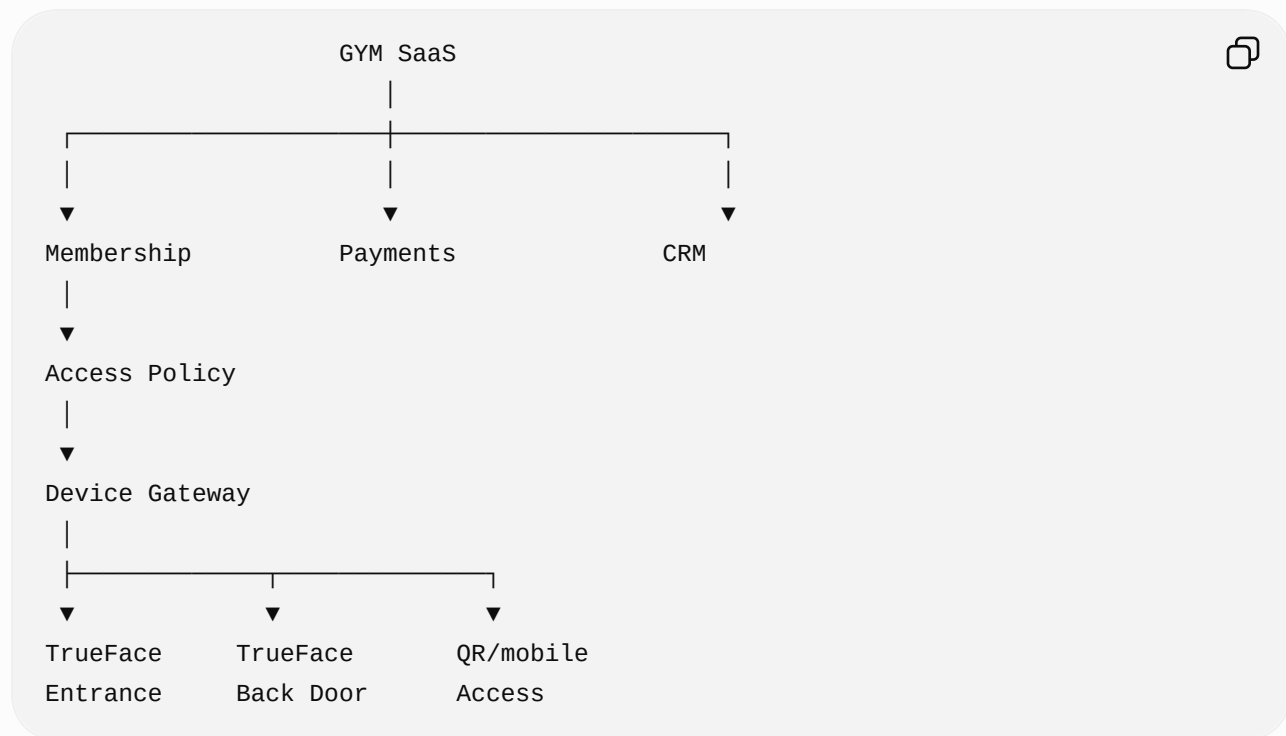
This is no longer just:

"Let's make a better iAS."

It can become:

"A modern membership/access-control platform that directly controls existing biometric hardware."

For example, for a gym:



And then:

Membership expires → access automatically expires.

Membership renewed → access automatically restored.

Membership frozen → access suspended.

Payment overdue → access restricted.

Employee terminated → biometric access revoked.

New member → biometric enrollment + authorization.

That's a genuinely useful system.

What I would do next

Since we now know from your datasheet that **this particular device has SDK/API support**, I would **not investigate packet sniffing or the iAS database yet**.

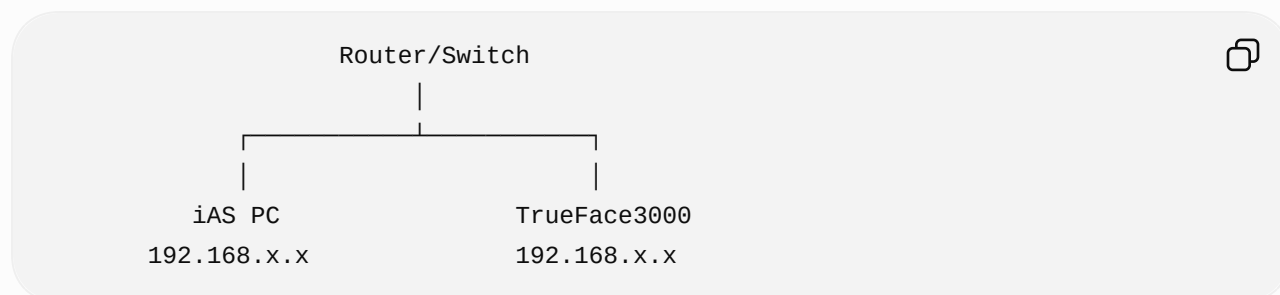
I'd make the next step:

Step 1 — Obtain the TrueFace3000 SDK/API

We need to inspect exactly what operations it exposes.

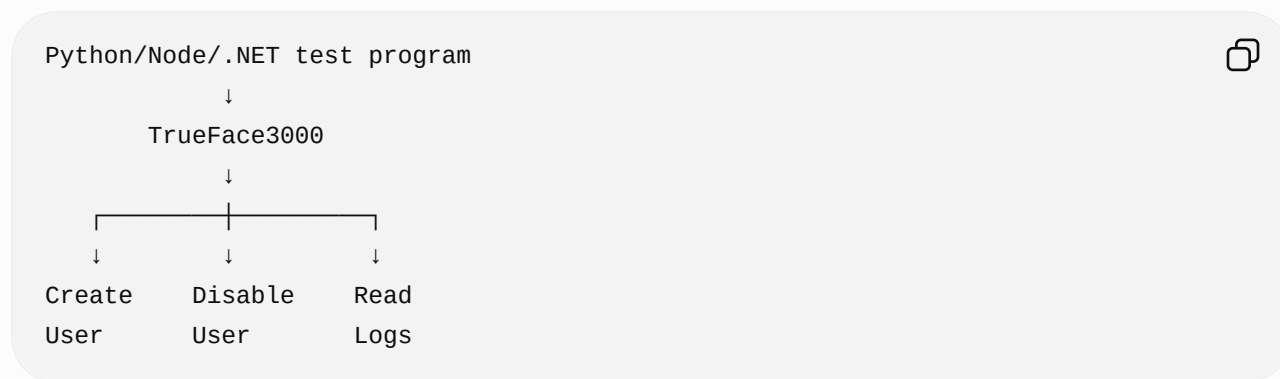
Step 2 — Determine the current network topology

Something like:



Step 3 — Build a tiny POC

Before building the SaaS:

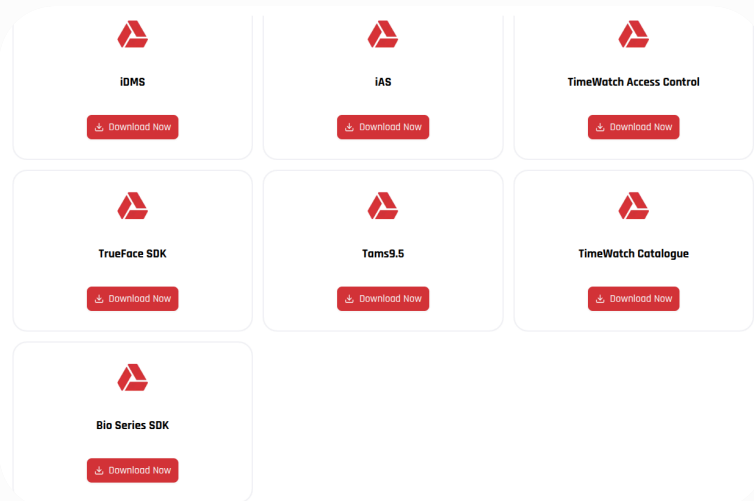


If we can successfully perform those operations, **we have essentially validated the core technical feasibility of your entire idea.**

And because the datasheet explicitly says **"SDK and API: Yes"**, **TCP/IP**, **Network Update**, **Remote Verification**, and **Restricted/Trusted List**, I think this POC is absolutely worth pursuing. [TrueFace3000-Datasheet-Ver.2.0 +1](#)

If you can get the **TrueFace SDK/API ZIP or documentation** from TimeWatch (the downloads page lists a TrueFace SDK), send it here. I can then examine the actual API and tell you **whether we can implement membership expiry/extension, user enable/disable, biometric enrollment, attendance retrieval, and device synchronization — and what the safest architecture would be.** [TimeWatch](#)

Sources



Can you tell which one to install? I don't have the hardware yet, can I still install and check what options are available in the sdk? how will it replace the existing iAS integrated flow? after the changes, can I expect that hardware would be integrated with my solution and not iAS? Also, if I manage the complete solution, will I also have to implement the face recognition checks? I don't want that.

Yes. And after seeing the actual **TrueFace3000 documentation + TimeWatch's download page**, I would take a fairly clear position:

Start with the TrueFace SDK, not iAS, not Bio Series SDK, and definitely not by reverse-engineering the network traffic.

TimeWatch's download page specifically lists a **TrueFace SDK**, separately from iAS, TimeWatch Access Control, and Bio Series SDK. [TimeWatch Downloads](#) ↗

I also followed the TrueFace SDK download link: it points to a `TrueFace_SDK.zip` hosted by TimeWatch's Google Drive. [TrueFace SDK ZIP](#) ↗

1. Which one should you install?

For your **TrueFace3000**, start with:

✓ TrueFace SDK

Not:

- × iAS — that's the legacy/application layer you're trying to move away from.
- × TimeWatch Access Control — likely their packaged access-control application, not what you want to build on top of.
- × Tams9.5 — unrelated/legacy software.
- × Bio Series SDK — potentially useful later for other biometric devices, but I would first determine what the TrueFace-specific SDK exposes.
- × iDMS — not your target.

The reason is particularly strong because your TrueFace3000 datasheet explicitly says:

SDK and API: Yes

and supports TCP/IP, network updates, remote verification, restricted/trusted lists, etc.

 TrueFace3000-Datasheet-Ver.2.0 +1

2. Can you install the SDK without having the hardware?

Yes — and I recommend doing exactly that.

You can download and inspect the SDK **before buying/connecting the hardware.**

But there is an important distinction:

You can inspect:

- DLLs / libraries
- header files
- example projects
- API classes/functions
- documentation
- sample code
- supported platforms
- communication methods
- device discovery mechanisms
- commands exposed by the SDK
- user-management functions
- attendance/log functions

- device configuration functions

You probably cannot fully test:

```
Connect to device
  ↓
Authenticate
  ↓
Create user
  ↓
Upload face
  ↓
Disable user
  ↓
Download attendance
```



without an actual compatible device.

So think of it as:

WITHOUT HARDWARE



```
SDK
|
|— What functions exist?      ✓
|— What APIs exist?          ✓
|— How do I call them?       ✓
|— What parameters?          ✓
|— Sample applications?      ✓
|— Can I build integration?   ✓
|
|— Does command actually work? ✗
    (needs hardware)
```

That's actually a **very good first POC**.

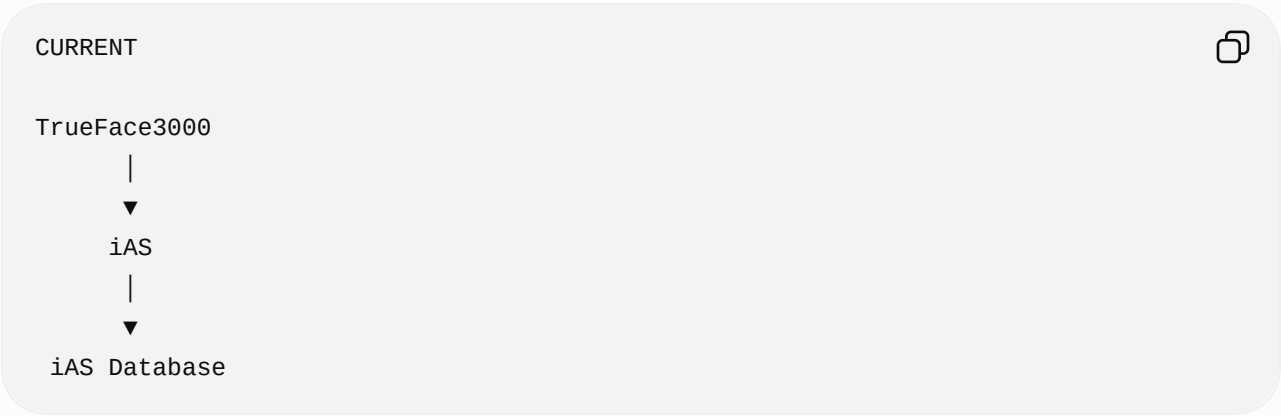
3. But here's the most important thing I want you to understand

You asked:

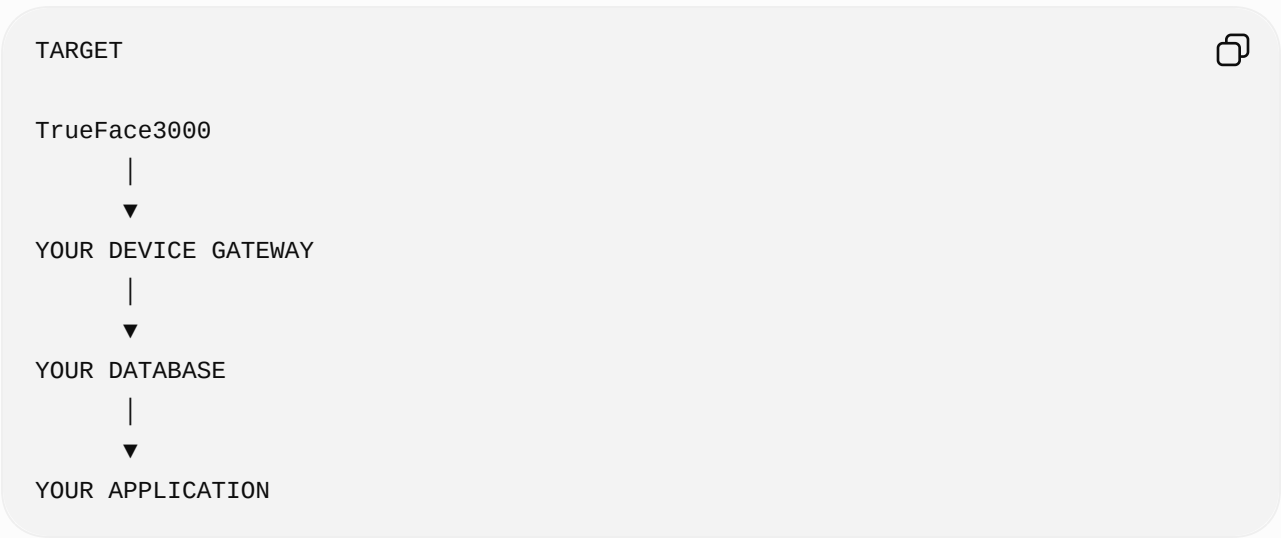
"How will it replace the existing iAS integrated flow?"

It doesn't happen by installing your software alongside iAS.

Eventually, you want to change this:



into:



iAS disappears from the operational path.

That's the ultimate goal.

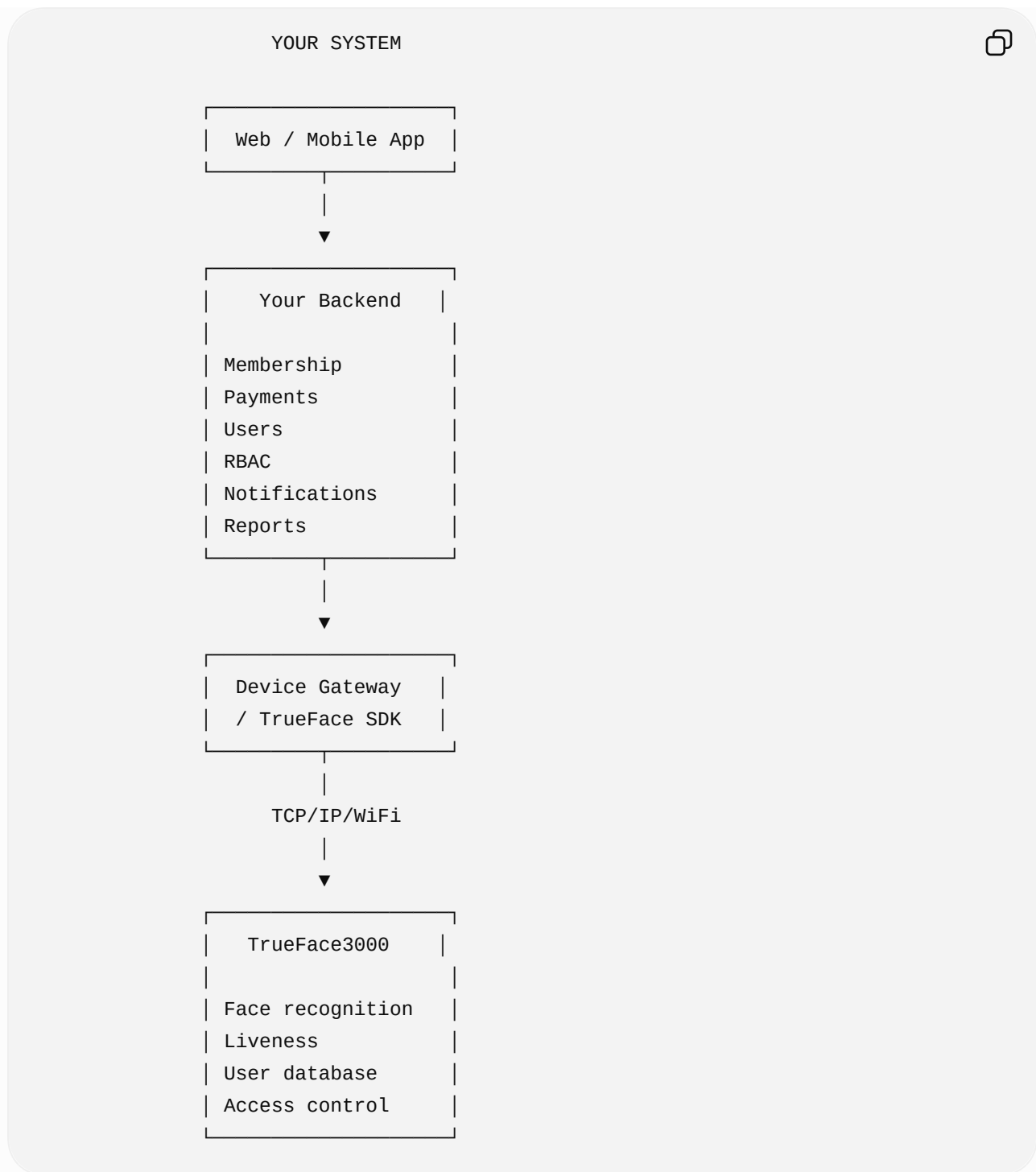
4. And YES — the hardware can potentially be integrated directly with your application

This is where the SDK/API becomes critical.

The TrueFace3000 is explicitly described as a **standalone** device, with TCP/IP/Wi-Fi, SDK/API support, network update, remote verification, and restricted/trusted lists.

 TrueFace3000-Datasheet-Ver.2.0 +1

So the desired architecture is:



And iAS isn't required in that final architecture.

5. Now your biggest question: do YOU have to implement face recognition?

No. This is actually the best part.

You should NOT implement face recognition.

You shouldn't need to build:

Camera



Image processing



Face detection



Face embedding



Face matching



Liveness detection



Identity



That's the device's job.

The TrueFace3000 already performs the facial recognition. The datasheet says it has:

- 2-MP dual-lens camera
- IR illumination
- liveness detection
- face recognition
- 99.9% stated accuracy
- 0.2-second 1:N comparison
- capacity for 5,000 faces.

TrueFace3000-Datasheet-Ver.2.0

So your system should treat the device as a **biometric engine**.

6. Think of it like this

You don't need to know **how** the device recognizes someone.

Your application just needs to say:

"Put user 1234 on this device."

The device handles:

User 1234



```
Face enrolled
↓
Face recognition
↓
Match?
↓
YES
```

Your application handles:

```
Is User 1234 allowed to enter today?
```



```
Membership:
01 Sep → 30 Sep
```

```
Today:
07 Sep
```

```
YES
```

But here's where we need to understand the SDK.

There are two possible designs.

7. Design A — Device handles authorization ★★★★★

This is what I **hope** the SDK supports.

Your system:

```
Member
ID: 1234
Valid until: 30 Sep
```



You send to device:

```
User = 1234
Valid From = 01 Sep
Valid Until = 30 Sep
Status = Active
```



The device stores it.

Then:

30 Sep



Membership expires



Your backend tells device



User disabled/expired



On 1 October:

Face recognized



User = 1234



Device says:

EXPIRED



Door remains locked



This is the architecture we want.

The device does face recognition **and local access authorization**.

8. Design B — Device only recognizes; YOUR server decides

Another possibility is:

Person



TrueFace



"User 1234 detected"



Your Server



"Membership active?"



YES / NO



This is also possible in some access-control systems.

But it introduces problems:

Internet down
↓
Server unreachable
↓
What happens?



For a gym, you don't necessarily want:

Internet failure → nobody can enter.

So **local device authorization** is preferable if the TrueFace SDK supports it.

The datasheet's "Restricted List/Trusted List", "Period eList", "Holiday Period", "Network Update", and "Remote Verification" make this worth investigating.

TrueFace3000-Datasheet-Ver.2.0

But **we cannot conclude from the datasheet alone exactly how those functions are exposed through the SDK.**

That's what we'll discover next.

9. Your membership extension becomes beautifully simple

Suppose:

Rahul
Membership:
01 Jan → 30 Sep



Your database:

membership_end = 2026-09-30



Device:

Rahul → active until 30 Sep



Then Rahul pays for another 3 months.

Your application:

Payment



Membership extension



31 Dec



Device synchronization



TrueFace



Now the device knows:

Rahul → active until 31 Dec



No iAS database manipulation.

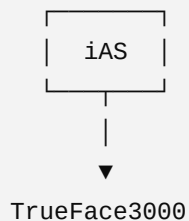
No dangerous SQL.

No manual intervention.

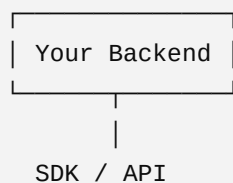
10. What happens when you completely replace iAS?

Eventually:

Today



Your new system



TrueFace3000

iAS can be shut down.

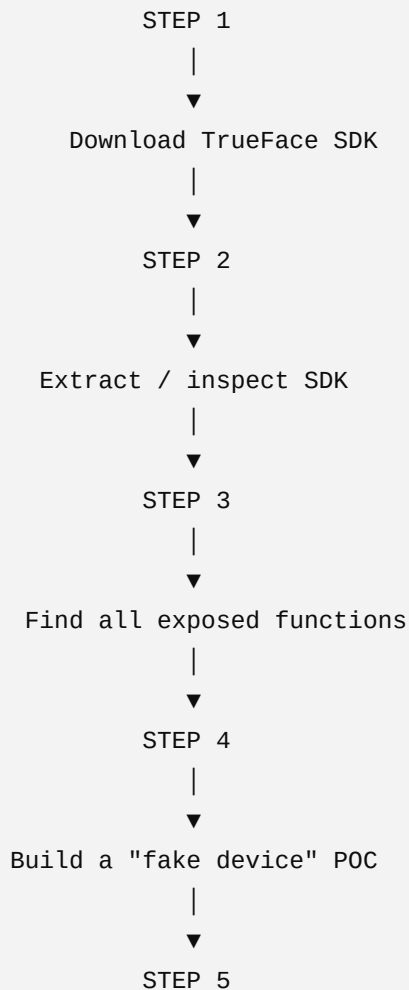
The device doesn't inherently need iAS just because it was previously configured to use iAS.

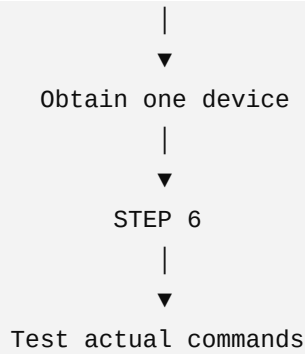
However: this is something we should verify with the actual SDK/manual before promising a production migration. TimeWatch explicitly markets the TrueFace3000 as a standalone device with its own network/software integration capabilities.  TimeWatch +1

11. There's one thing I would NOT do yet

Don't buy the hardware and immediately start coding the entire SaaS.

First do this:





12. What I specifically want to find in the SDK

When you download/extract `TrueFace_SDK.zip`, don't worry about understanding everything.

Send me the extracted folder/ZIP.

I'll want to identify whether we have functions resembling:

Device

- |— Connect()
- |— Disconnect()
- |— GetDeviceInfo()
- |— GetStatus()
- |— Reboot()

User

- |— CreateUser()
- |— UpdateUser()
- |— DeleteUser()
- |— EnableUser()
- |— DisableUser()

Face

- |— EnrollFace()
- |— DeleteFace()
- |— UpdateFace()

Access

- |— SetAccessRule()
- |— SetValidity()
- |— SetSchedule()
- |— SetPermission()

Attendance

```
|— GetLogs()  
|— GetAttendance()  
└— ClearLogs()
```

Device Sync

```
|— PushUser()  
|— PullUser()  
└— Sync()
```

The exact names obviously may be completely different.

But **these capabilities are what matter**, not whether the SDK has a pretty UI.

13. One more distinction: enrollment

You said:

"I don't want to implement face recognition checks."

You shouldn't.

But your application **may need to implement face enrollment workflow**.

That's different.

For example:

```
Admin opens your app  
  ↓  
"Add Member"  
  ↓  
Name / Phone / Membership  
  ↓  
"Enroll Face"  
  ↓  
Device enters enrollment mode  
  ↓  
Member looks at TrueFace  
  ↓  
TrueFace creates biometric template  
  ↓  
Template stored on device  
  ↓  
Your system stores device-user mapping
```



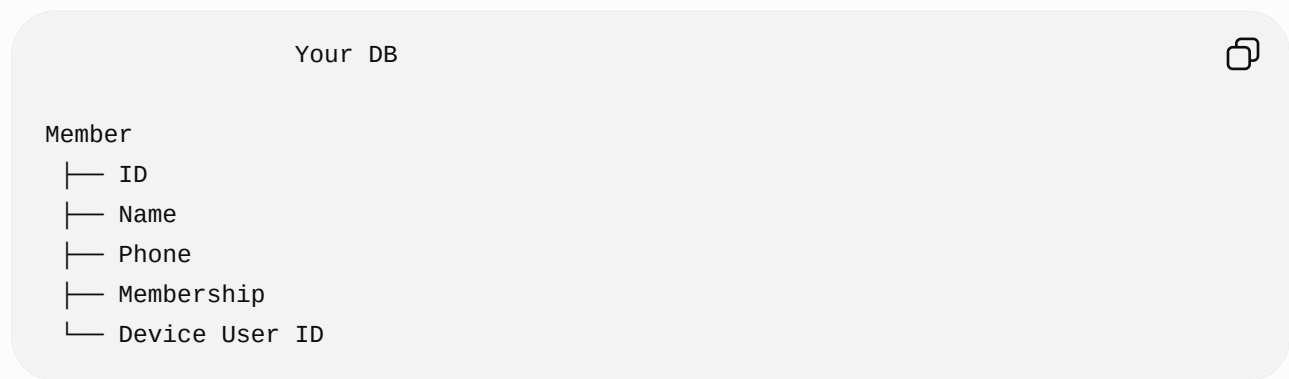
Your software **orchestrates** the process.

The device performs the actual biometric processing.

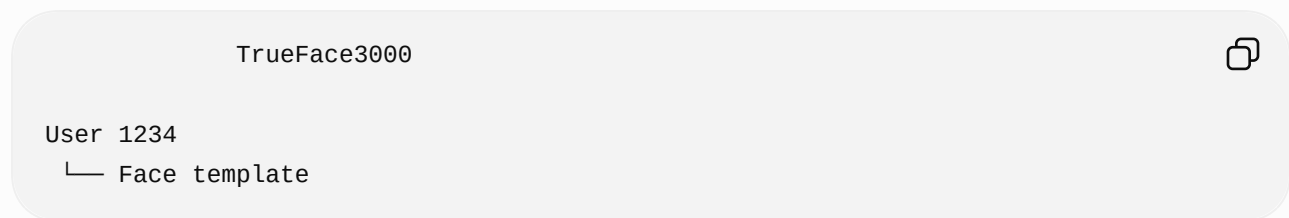
That's the ideal separation.

14. And I'd actually avoid downloading/storing face images

Ideally:



while:



Your backend doesn't need to possess the person's facial biometric template unless the SDK/integration architecture makes that unavoidable.

That keeps your system considerably simpler from a security/privacy standpoint.

Bottom line

Yes, download/install the TrueFace SDK now, even without the hardware.

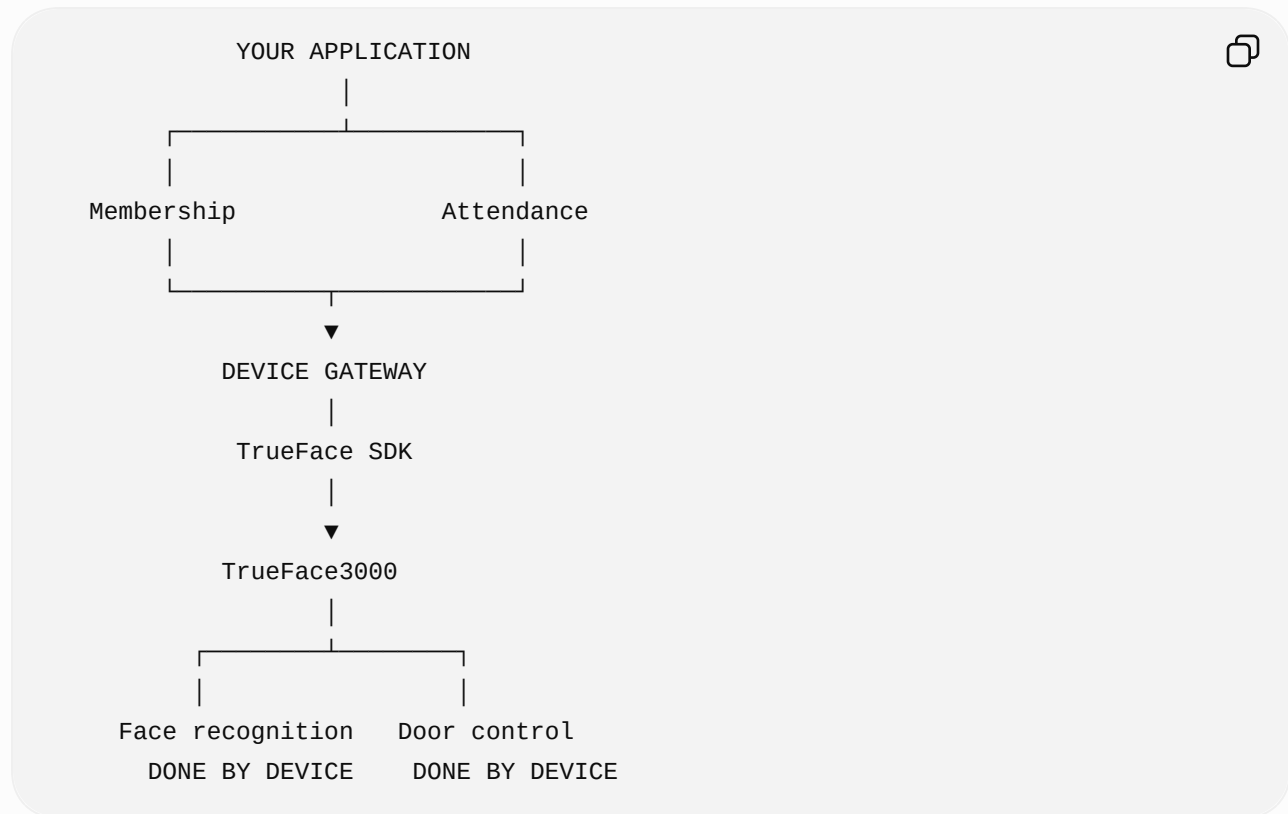
Your first objective isn't to make it work. It's to answer:

"What can this SDK actually control?"

And the fact that TimeWatch separately provides a **TrueFace SDK**, while the TrueFace3000 specification explicitly says **SDK/API = Yes**, is a very encouraging sign.

 TimeWatch +1

Your eventual architecture should ideally be:



You do not implement facial recognition.

You do not need iAS in the final architecture.

Your application becomes the source of truth for membership/access.

The TrueFace remains responsible for biometric recognition.

Best next step

Download [TrueFace_SDK.zip](#) **and upload the ZIP/folder here.** I can inspect the SDK itself and tell you, concretely, whether it appears to support the critical operations we need — especially **create/disable users, validity dates, face enrollment, access permissions, attendance retrieval, and device synchronization** — before you spend money on the hardware.

Sources





NetSDK Programming ...
PDF

```
~/Downloads/TrueFace_SDK$ ls
AccessDemo2s    NetSDKCS
AccessDemo2s.sln 'NetSDK Programming Manual
(Intelligent Buliding).pdf'

AccessDemo2s.csproj  bin    OpenDoorEventForm.cs
QueryRecord
AccessForm.cs        Common.cs
OpenDoorEventForm.Designer.cs Setting
AccessForm.Designer.cs Config
OpenDoorEventForm.resx  UserManager
Show more ▾
```

Worked for 18s >

This is **very good news**. The SDK/manual you have is much more capable than a simple attendance SDK. In fact, after reading the manual, I would move the project from "possible" to **"technically very promising, but we need to verify TrueFace3000 compatibility."**

The most important discovery is that this SDK provides exactly the kinds of device-side operations your architecture needs.

1. What we have confirmed

The SDK is a C# wrapper around **NetSDK**, with native libraries such as `dhnetSDK.dll` / `libdhnetSDK.so`. The manual says the SDK can communicate with devices, remotely control them, query device data, and configure device data. It supports Windows and common Linux systems.  NetSDK Programming Manual (Inte...

More importantly, for the **second-generation access-control/face devices**, the SDK exposes:

User management

It can:

- query users
- add users
- change users
- delete users
- clear users

and the user information includes:

user ID, person name, type, status, ID card number, valid period, holiday plan, and user permission.  NetSDK Programming Manual (Inte...

That's **almost exactly what we wanted for membership management.**

2. Membership expiry/extension looks especially promising

The SDK documentation explicitly deals with **valid start time** and **valid end time** for access-control users/cards.  NetSDK Programming Manual (Inte...

The sample code also reads/writes:

```
ValidStart  
ValidEnd
```



from the device.  NetSDK Programming Manual (Inte...

So our desired operation:

```
Membership:  
01 Sep → 30 Sep
```



can potentially become:

```
TrueFace user:  
ValidStart = 01 Sep  
ValidEnd   = 30 Sep
```



Then:

Membership renewed



Your database



SDK



Update device



ValidEnd = 31 Dec



This is precisely the capability we were hoping to find.

3. And we can manage the actual face data

This is even more significant.

The SDK has a dedicated:

Face Management

module.

It explicitly supports adding, deleting, querying and modifying face information, including the user ID and face picture. [NetSDK Programming Manual \(Inte...](#)

The available operations are:

INSERT

GET

UPDATE

REMOVE



through:

`NETClient.OperateAccessFaceService`

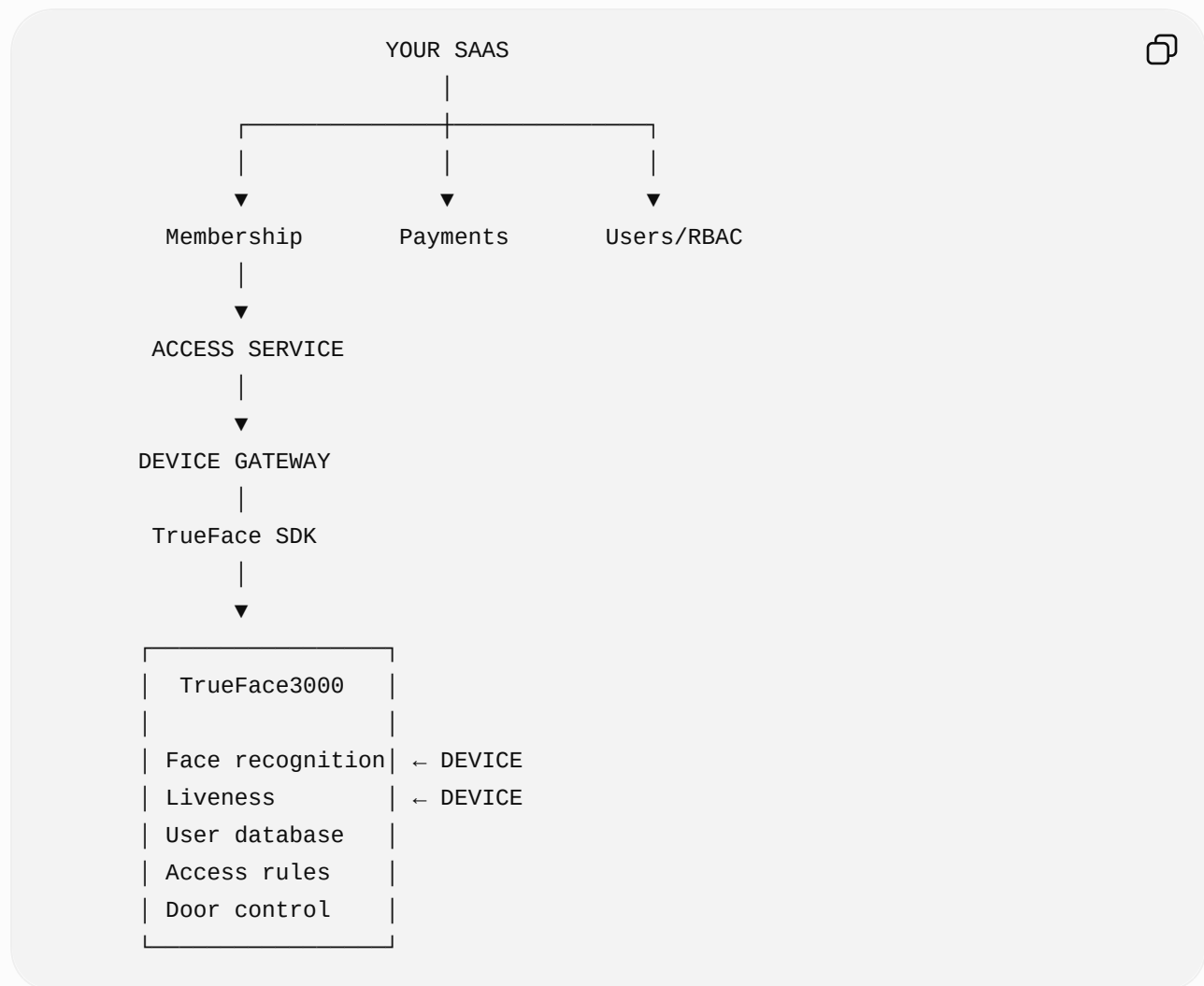


[NetSDK Programming Manual \(Inte...](#)

And the sample actually demonstrates retrieving a face image from the device and inserting a face image into the device. [NetSDK Programming Manual \(Inte... +1](#)

This means your application **does not need to implement facial recognition.**

4. This is the architecture I'd now target



Your software manages **who should have access**.

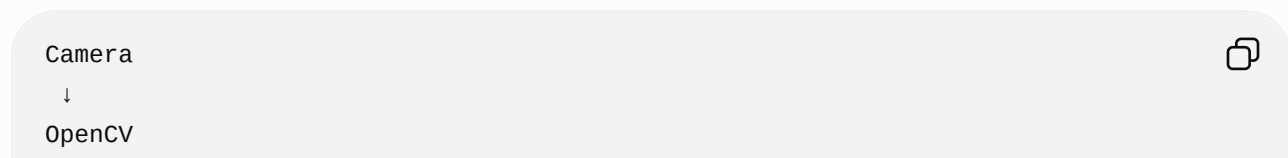
The device manages **whether the person's face matches**.

That's a very clean separation.

5. You don't implement face recognition

This is worth emphasizing because you specifically said you don't want to.

You do **not** need:



↓
 Face detection
 ↓
 Face embedding
 ↓
 Face matching
 ↓
 Liveness

The device already does that.

Your system might simply send:

User ID: 12345
 Face: [image]



to the device.

The device handles the biometric processing.

The SDK's face-management API is concerned with managing the face information on the access-control device; the manual does **not** ask the application to implement the recognition algorithm itself. [NetSDK Programming Manual \(Inte...](#)

6. We can also retrieve access records

The SDK has a records-query module for access/alarm records.

[NetSDK Programming Manual \(Inte...](#)

So we can potentially have:

TrueFace

```

  |
  ├── User/face data
  ├── Authorization
  └── Access records
      |
      ▼
    Your Gateway
      |
      ▼
    Your Database
  
```



That means your database can contain the complete attendance/access history without relying on the iAS database.

7. We can also receive events

The SDK supports callbacks for:

- device discovery
- disconnection
- reconnection
- real-time monitoring
- alarms

The manual specifically documents disconnect/reconnect callbacks.

 NetSDK Programming Manual (Inte...

This is important because our gateway can know:

Device online ✓
Device offline ⚠
Device reconnected ✓



and automatically synchronize.

8. There's also direct door control

The SDK can remotely open and close the access-controlled door and query door state.

 NetSDK Programming Manual (Inte...

So eventually your application could potentially have something like:

Open Gate

for an authorized administrator.

But this should be heavily protected with RBAC + audit logging because it is a physical-security operation.

9. BUT — there is one critical issue

We cannot yet say that this exact SDK definitely works with your TrueFace3000.

This manual's supported-device list contains various access-control devices such as:

```
ASI1202M
ASI7213X
ASI7213Y-D
ASI7214X
...
```



under second-generation access control.

NetSDK Programming Manual (Inte...

TrueFace3000 is not explicitly listed in the portion of the manual we have.

That doesn't necessarily mean it won't work.

The TrueFace3000 could be a TimeWatch-branded/rebadged device based on the same underlying platform/protocol, but **we should not assume that**.

This is now the #1 thing we need to establish.

10. And your folder structure gives us another clue

You have:

```
AccessDemo2s
AccessDemo2s.sln
AccessForm.cs
OpenDoorEventForm.cs
QueryRecord
Setting
UserManager
Fingerprint
NetSDKCS
```



That's excellent.

Especially:

```
UserManager
QueryRecord
```



```
AccessForm
OpenDoorEventForm
NetSDKCS
```

These are exactly the areas I would inspect.

I would expect something roughly like:

```
UserManager/
  user CRUD
  face management
  card management

QueryRecord/
  attendance/access logs

AccessForm/
  access control

OpenDoorEventForm/
  door events

NetSDKCS/
  C# wrapper
  structures
  enums
```



11. Don't modify the SDK yet

Right now, **don't change anything in the downloaded SDK.**

Instead, let's first reverse-engineer **the capabilities**, not the protocol.

Since you're on Linux, from the SDK directory you can run:

```
</> Bash

find . -maxdepth 3 -type f | sort
```



Then:

```
</> Bash
```



```
grep -RniE \  
'Face|User|Valid|Access|Record|Insert|Update|Remove|Delete|Card|OperateAccess' \  
--include='*.cs' .
```

And:

</> Bash

```
grep -RniE \  
'NET_ACCESS_USER_INFO|NET_ACCESS_FACE_INFO|OperateAccessFace|InsertOperateAccessUser|'  
--include='*.cs' .
```

That will give us a **much faster** picture of the actual functions exposed by the SDK.

12. Even better: upload the SDK

If possible, zip:

```
TrueFace_SDK/
```

and upload it here.

Then I can inspect:

```
NetSDKCS/  
userManager/  
QueryRecord/  
AccessForm/  
OpenDoorEventForm/  
Config/
```

and specifically map:

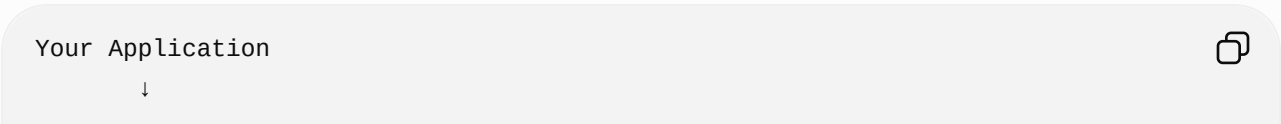
Requirement	SDK capability
Create member on device	?
Delete member	Yes
Enable/disable member	?
Membership expiry	Very promising

Requirement	SDK capability
Extend membership	Very promising
Face enrollment	Yes, face management
Face recognition	Device-side
Retrieve attendance	Yes, records query
Real-time access event	?
Door open/close	Yes
Door status	Yes
Multiple devices	Likely
Device online/offline	Yes
Device synchronization	Need to implement
Mobile app	We build
RBAC	We build
Notifications	We build
Billing	We build
Membership logic	We build

That will give us a **real integration specification** rather than guessing from marketing material.

13. One architectural correction from my previous answer

Earlier I suggested a possible:



```
graph TD
    A[Your DB] --> B[Device Gateway]
    B --> C[TrueFace]
```

I'd now make that more precise:

```
graph TD
    A[YOUR DATABASE] --> B[membership state]
    B --> C[ACCESS SERVICE]
    C --> D[desired device state]
    D --> E[DEVICE GATEWAY]
    E --> F[NetSDKCS]
    F --> G[TrueFace3000]
```

The **Device Gateway** should be the only component allowed to modify the hardware.

So if an employee changes:

Membership expiry: 30 Sep → 31 Dec

the request goes:

```
graph TD
    A[Mobile/Web] --> B[Backend]
    B --> C[Membership Service]
    C --> D[Database transaction]
    D --> E[Device command]
    E --> F[TrueFace]
    F --> G[ACK]
```

↓
Synchronization recorded

This gives us a very reliable system.

14. The one thing I want to avoid

Don't make this:

Your Application
↓
TrueFace

AND

iAS
↓
TrueFace



with both systems controlling users.

Once we go live, **one system must be authoritative.**

Ideally:

YOUR SYSTEM
|
▼
TrueFace



and iAS is removed from the operational chain.

During migration, we can temporarily run:

TrueFace
↗ ↖
iAS Your Gateway



but we should be extremely careful about simultaneous writes.

My current assessment

Based on the SDK manual you uploaded:

Technical feasibility: ● HIGH

Membership expiry/extension: ● VERY PROMISING

User management: ● CONFIRMED in SDK

Face management: ● CONFIRMED in SDK

Attendance/access records: ● CONFIRMED in SDK

Door control: ● CONFIRMED in SDK

Need to implement face recognition: ● NO

TrueFace3000 compatibility: ● MUST VERIFY

The manual itself says it is a reference and that actual product behavior can differ, and recommends contacting the supplier/customer service for the latest program and supplementary documentation. [NetSDK Programming Manual \(Inte...](#)

So don't buy the hardware yet.

Upload the complete `TrueFace_SDK` directory as a ZIP. That's now the most useful thing you can give me. I'll inspect the actual `UserManager` , `QueryRecord` , `AccessForm` , `NetSDKCS` code and tell you **exactly what we can build, what is missing, and whether we can make your application completely independent of iAS.**

Sources



ChatGPT can make mistakes. Check important info.